

Introduction to Atomistic Simulations with MLIP *(Machine-Learned Interatomic Potential)*

Fadjar Fathurrahman

29 June 2026

The Atomistic View of Matter

- Matter is represented by atoms and their interactions
- Important quantities:
 - Atomic positions
 - Forces
 - Energies
 - Electronic states
- Goal:
 - Understand how atomic arrangements determine material properties

What is atomistic simulation?

- Computer-based modeling of materials and molecules at the nanoscale
- Predicts structure, properties, and dynamics using physical models
- Bridges the gap between quantum mechanics and experiments

Some simulations techniques:

- Geometry optimizations
- Vibrational analysis
- Saddle point searches
- molecular dynamics
- Monte Carlo simulation

In order to model interactions between atoms, we need *interatomic potentials* or *force fields*.

Interatomic Potentials

- Describe how atomic energy depends on atomic arrangement

General form:

$$E = E(R_1, R_2, \dots, R_N)$$

where:

- R_i = atomic positions
- E = potential energy

Forces can be obtained by taking derivative of E w.r.t atomic positions.

Examples:

- *Ab-initio*-based potentials (derived from electronic structure calculations: density functional theory, Hartree–Fock, etc)
- Empirical interatomic potentials: using simplified analytic form, need many a-priori assumptions about interactions between atoms, examples Lennard-Jones potential, Embedded Atom Method (EAM), Tersoff potential, etc.
- *Machine-learned interatomic potentials*: recent development, using data derived from electronic structure calculations, making very few assumptions about interactions between atoms.

The need for machine-learning interatomic potentials

- *Ab-initio*-based potentials are accurate but need large computational resources.
- Empirical interatomic potentials don't need large computational resources but its accuracy is limited
- *Machine-learned interatomic potentials* (MLIPs): reside somewhat between *ab-initio*-based and empirical interatomic potentials. In recent years there are explosive development of MLIPs, involving not only academic labs but also research labs by large corporations such as Google, Facebook, and Microsoft.

How MLIPs are developed

- Need training data
 - early MLIPs usually uses training data tailored for specific systems or applications
 - parameters are fitted to this data, typically using non-parametric or neural network models, thus are more generalizable than empirical interatomic potentials
- Recent MLIPs uses very large dataset, for examples:
 - Materials Project (<https://next-gen.materialsproject.org/>)
 - Open Catalyst Project (OC20/OC22)
 - MAD (Massive Atomic Diversity)
 - OMol25
 - CatBench
 - ...

Foundation models

Recent development in MLIPs have given rises to several universal MLIPs or *foundation models* for atomistic simulations.

In this context, “universal” means the model acts as a “foundation potential”. Instead of being trained for one specific material or molecule, it is pre-trained on massive datasets covering most of the periodic table. This allows a single model to accurately simulate almost any chemical system, organic material, or inorganic compound.

Advantages:

- Broad Applicability: we can use it on metals, polymers, liquids, or gases without needing to train a new, customized model from scratch.
- Versatility: While it provides an excellent baseline right out of the box, researchers can also *fine-tune* universal MLIPs to highly specific, complex atomic environments

Some foundation models

- MACE (<https://github.com/acesuit/mace>)
- CHGNET (<https://chgnet.lbl.gov/>)
- MatterSim (<https://github.com/microsoft/mattersim>)
- FairChem (<https://github.com/facebookresearch/fairchem>)
- GNoME (https://github.com/google-deepmind/materials_discovery)
- SevenNet (<https://github.com/mdil-snu/sevennet>)
- UPET (<https://github.com/lab-cosmo/upet>)

We will use MACE in this tutorial.

Practical Session

What are needed?

Hardware:

- A PC or laptop with internet connection

Softwares:

- Python, JupyterLab, Internet Browser
- Python libraries: ASE, WeasWidget, MACE, UPET
- Optional: Avogadro, Xcrysden, VESTA, or other molecular/atomistic visualization softwares

Other:

- Google account (if using Google Colab)

What will be done?

- Main objective:

Calculating adsorption energy of a molecule adsorbed on a metal surface

$$E_{\text{ads}} = E_{\text{surf-ads}} - (E_{\text{surf}} + E_{\text{mol}}) \quad (1)$$

- To calculate this quantity, we need to do several things:
 - to construct a model for metal surface, i.e. a *slab*, such as fcc(111), bcc(110), hcp(0001) and others
 - to construct a model for molecule (or atom)
 - to construct a model for slab + molecule
 - to relax the geometries of each to these structures and obtain the energy of the relaxed geometries

We can do this for several kinds of surfaces and molecules.

```
pip install ase mace-torch
pip install weas-widget
!curl -L -J -O https://github.com/ACEsuit/mace-foundations/releases/download/mace_mh_1/
mace-mh-1.model
```

Preparing the Calculator for MLIP Model

We will use MACE (<https://github.com/acesuit/mace>) for MLIP.

```
import torch
device = "cuda" if torch.cuda.is_available() else "cpu"
from mace.calculators import MACECalculator
model_path = "./mace-mh-1.model" # change this with the actual location
calc = MACECalculator(
    model_paths=model_path, device=device, head="oc20_usempbe"
)
```

Download the model with command line within the Jupyter notebook

```
!curl -L -J -O https://github.com/ACESuit/mace-foundations/releases/download/mace_mh_1/
mace-mh-1.model
```

or manually using the browser

In Google Colab we can use T4 GPU, please change the runtime to GPU.

Dealing with atomistic structure and dynamics

We will use ASE (Atomic Simulation Environment) (<https://ase-lib.org/>) to handle (create, manipulate, e.t.c) atomistic structures.

We will also use algorithms provided in ASE to relax or optimize the structures.

Creating Atoms object

Atoms is the central object in ASE. It is used to hold many information about atomistic structure such as composition (atom types), coordinates, lattice vectors (for periodic systems), etc.

An Atoms object can be initialized like this:

```
from ase import Atoms
atoms = Atoms('OHH', positions=[
    (0, 0, 0),
    (0, 0, 0.9),
    (0, 1.1, 0.0)
])
```

ASE uses angstrom (for length) and eV (energy) as default units.

We also can do many things with Atoms object. Please see the ASE documentation at <https://ase-lib.org/ase/atoms.html>.

Creating Atoms object from external files

One alternative way to construct Atoms object is to use existing database of atomistic structures:

- <https://www.crystallography.net/cod/>,
- <https://pubchem.ncbi.nlm.nih.gov/docs/structure-search>,
- <https://next-gen.materialsproject.org/>

You can download the structure in various file formats such as .xyz, .cif, .pdb, e.t.c

We can read the structures use ase.io module:

```
import ase.io
atoms = ase.io.read("structures.xyz") # or cif or pdb
```

We also can use molecular editor such as Avogadro, ChemCraft or other to create the structure and save it to supported file formats.

ASE can read (and also write) many file types. Please see the documentation at <https://ase-lib.org/ase/io/io.html>.

Creating Atoms object with built-in utility functions

ASE also provides many built-in utility functions to create various common atomistic structures. Please see the documentation at <https://ase-lib.org/ase/build/build.html>.

For example, we can create H₂O structure with the following code

```
import ase.build  
atoms = ase.build.molecule("H2O")
```

Visualisasi

There are many ways to visualize atomistic structures, either via ASE or external viewers/visualizers.

We will cover only some ways that we may use in this course.

ASE provides view function from `ase.visualize` module

```
from ase.visualize import view
view(atoms) # GUI
view(atoms, viewer="x3d") # notebook
```

The also can be called via terminal using ase command line:

```
ase gui H2O.xyz
```

Visualisasi dengan WeasWidget

WeasWidget (<https://weas-widget.readthedocs.io/>) provides interactive viewer that can be used in Jupyter notebook.

```
from weas_widget import WeasWidget
widget = WeasWidget()
widget.from_ase(my_atoms) # change my_atoms to your Atoms object
widget
```

WeasWidget also can be used to edit structure.

Using Matplotlib

We also can use use Matplotlib for quick and non-interactive visualization. For example:

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(4, 4))
plot_atoms(atoms, ax=ax, rotation='45x,10y,0z', scale=1.0)
plt.axis("off")
plt.savefig("IMG_atoms.png", dpi=300, bbox_inches="tight") # save to file
```

Try to change rotation into something like 90x,45y,30z and see the difference.

Using external visualization tools

Sometimes we are not satisfied with the available visualization and want to use our favorite tools. For example, my favorite is Xcrysden <http://www.xcrysden.org/>¹.

We can export or save to the suitable format and then open it with the external tools.

```
atoms.write("my_atoms.xyz") # for Avogadro, etc.  
atoms.write("my_atoms.xsf") # for Xcrysden, VESTA, etc.  
atoms.write("POSCAR") # for VASP
```

¹Sadly, it is not available natively on Windows, although we can access it via WSL (Windows Subsystem for Linux).

TASK 1

Please create a molecule and visualize it (using some ways that are described previously).

Geometry optimization of molecule

In structural relaxation (geometry optimization), the energy is minimized by changing the atomic positions until the minimum energy is achieved. This can be done using several algorithms, see <https://ase-lib.org/ase/optimize.html>.

Geometry optimization requires *energy* and *force* information from the atomistic structure. This information is calculated by Calculator object. We will use the MACE calculator.

In this example, we will use LBFGS algorithms

```
atoms.calc = calc # set the calculator
ftraj = "geoopt.traj" # please change with your preferred name
opt = LBFGS(surf, trajectory=ftraj) # for visualization
opt.run(fmax=0.05, 1000) # max force 0.05, max no. of iterations 1000
E_surf = surf.get_total_energy() # get the last/minumum energy
```

Trajectory file visualization

We can visualize the result of geometry optimization using WeasWidget in Jupyter notebook

```
atoms_traj = ase.io.read("geopt.traj", ":") # please change with your trajectory
filename
widget = WeasWidget()
widget.from_ase(atoms_traj)
widget
```

We also can use this command in the terminal:

```
ase gui geopt.traj
```

We also can convert trajectory file to other file format that can be visualized using external visualization tools. This command will convert the .traj file to .xyz file that can be visualized using, e.g., Avogadro.

```
ase convert geopt.traj@: geopt_traj.xyz
```


TASK 2

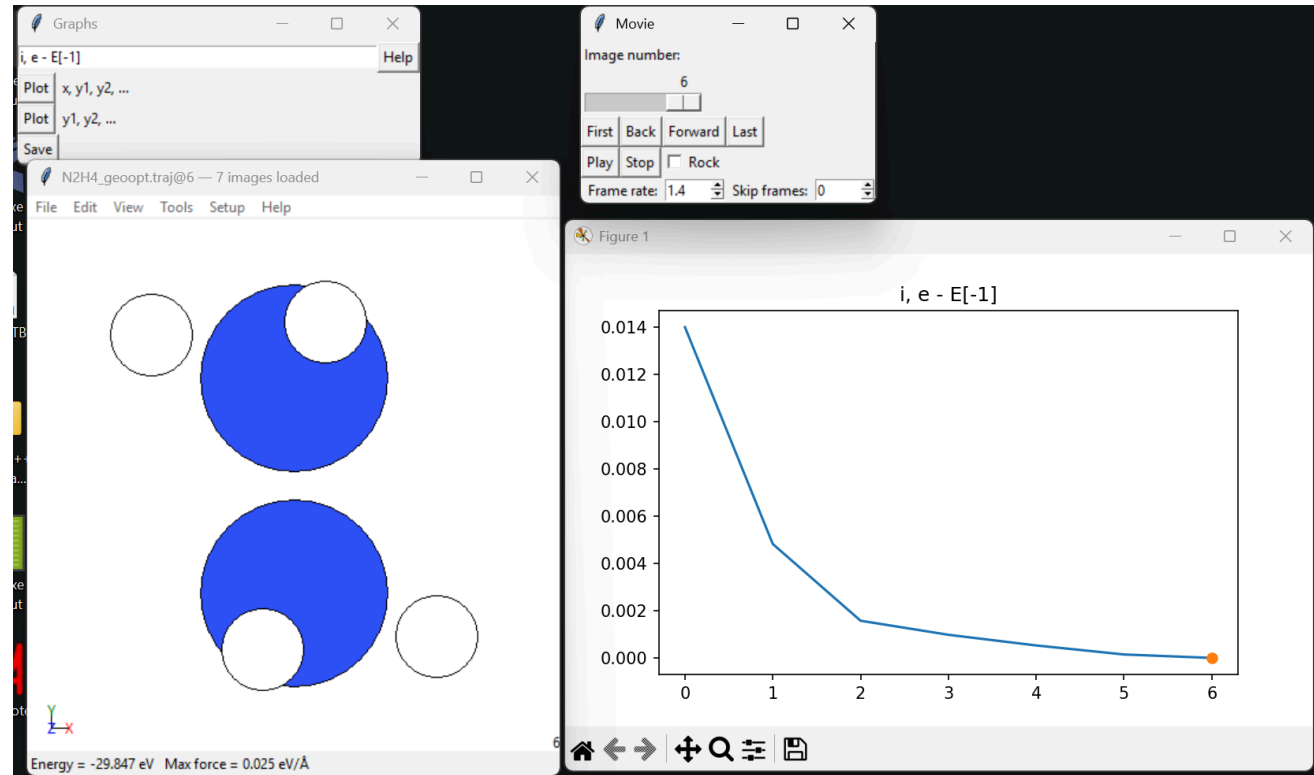
Buat molekul lain (yang akan diadsorpsikan ke permukaan) dan lakukan optimisasi geometri.

Do geometry optimization of molecule of your choice. For example, this code will create hydrazine, N_2H_4 , molecule and optimize its structure.

```
mol_isolated = ase.build.molecule("N2H4")
mol_isolated.calc = calc
ftraj = "N2H4_geoopt.traj" # use your preferred name
opt = LBFGS(mol_isolated, trajectory=ftraj)
opt.run(0.05, 1000)
E_mol_isolated = mol_isolated.get_total_energy()
```

Try to visualize the result of the geometry optimization

Example result (using ase gui)



Creating fcc(111) slab

We also can use ASE to create a slab model.

In this example we will create a Ni(111) slab, which have fcc(111) surface structure. We will use 4×3 supercell size and 5 layers. We also add vacuum of 10 angstrom between the periodic slab in z -direction.

```
import ase.build
from ase.visualize import view
surf = ase.build.fcc111("Ni", (4, 3, 5), vacuum=10.0)
view(surf, viewer="x3d") # for Jupyter notebook
```

Try to visualize the surface.

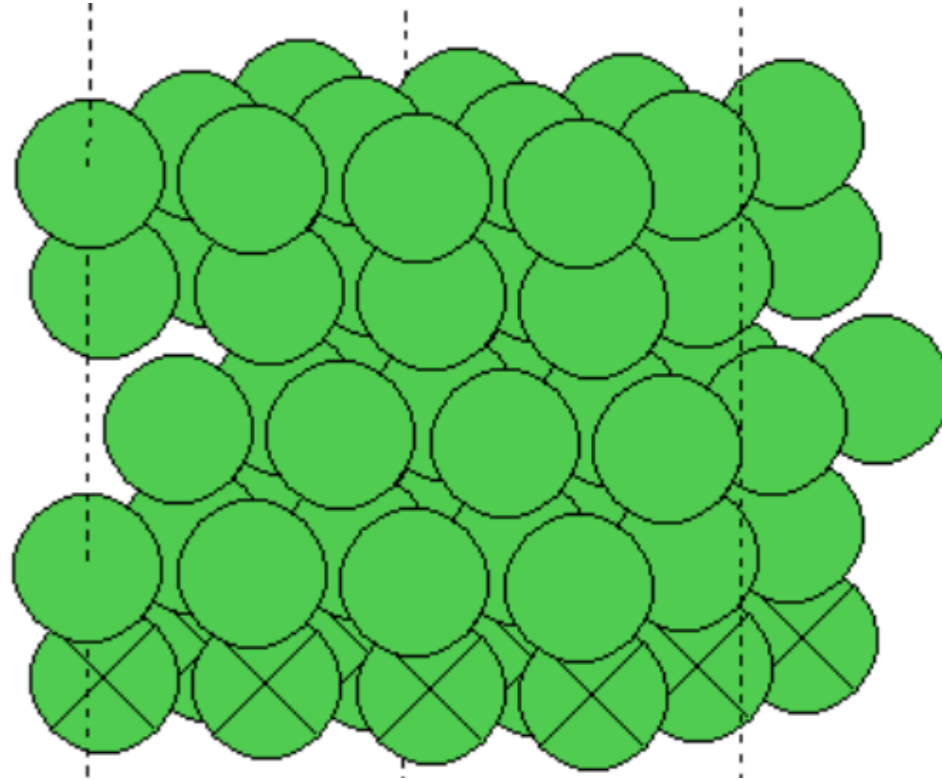
Try to change to (4,3,5) to other values and visualize the results.

Geometry optimization of slab

For surface cases, some atoms are typically kept fixed or fixed during relaxation. We will keep the atoms in the bottom layer of the structure immobile during relaxation:

```
zmin = surf.get_positions()[ :, 2].min() # Get the lowest layer z-coordinate
idx_atoms_lowest_layer = surf.positions[ :, 2] < zmin + 0.01 # Fix the lowest layer
constraint = FixAtoms(mask=idx_atoms_lowest_layer)
surf.set_constraint(constraint)
```

Ni(111) 3×4 with lowest layer fixed



Geometry optimization of slab

The code for geometry optimization is similar to the one we use for molecule.

```
surf.calc = calc
ftraj = "Ni111_surf_geoopt.traj" # please change with your preferred name
opt = LBFGS(surf, trajectory=ftraj)
opt.run(fmax=0.05, 1000)
E_surf = surf.get_total_energy()
```

Creating slab with adsorbate/molecule

Creating surface structures (slabs) with adsorbed molecules is generally more complicated, especially for complex surface structures (non-flat, with multiple features or candidate adsorption sites) and molecules with multiple atoms, allowing for different adsorption configurations.

We can use ase gui or a molecular editor like Avogadro to prepare the surface structure with the adsorbate/molecule. It is usually more convenient to set initial position of the adsorbate using GUI tools.

In the following Python code, we add mol_isolated to the surface at coordinates (3,3) and a height of 2.5 angstroms from the surface.

```
surf_with_ads = surf.copy()  
ase.build.add_adsorbate(  
    surf_with_ads, mol_isolated, 2.5, position=(3,3), mol_index=0  
)
```

You may change the height and position accordingly.

Geometry optimization of surface with adsorbate

The code is also similar

```
surf_with_ads.calc = calc
ftraj = "Ni111_N2H4_geoopt.traj" # please change with your preferred name
opt = LBFGS(surf_with_ads, trajectory=ftraj)
opt.run(fmax=0.05, 1000)
E_surf_with_ads = surf_with_ads.get_total_energy()
```

Visualize the result of geometry optimization using ase gui

```
ase gui Ni111_N2H4_geoopt.traj
```

You also can use other tools.

Adsorption energy calculation

Now we can calculate the adsorption energy as

$$E_{\text{ads}} = E_{\text{surf_with_ads}} - (E_{\text{surf}} + E_{\text{mol_isolated}})$$

A negative adsorption energy indicates that adsorption is exothermic.

The more negative the adsorption energy, the stronger the molecule/adsorbate is adsorbed to the surface.

For MACE, this energy value is usually not very accurate, so fine-tuning is necessary using new training data. However, I found that the final geometry is usually good.

If we want to use DFT, we usually need to carry out the above calculations at computer clusters, however, using MLIP (MACE) we can compute this on a laptop!

EXPLORATION TASK

Typically, the minimization algorithm used to find the adsorption configuration is a local minimum search algorithm, which relies on guesses or initial configurations of the surface and adsorbate structures. Typically, the adsorption structure optimization search is repeated using several guesses of the initial configurations.

In the following example we rotate the hydrazine molecule before adsorbing it to the surface.

```
surf_with_ads = surf.copy()
mol_isolated.rotate(90, "z", center="COM")
ase.build.add_adsorbate(
    surf_with_ads, mol_isolated, 2.5, position=(4,2.5), mol_index=0)
```

You also can use external GUI tools to edit the surface, export the results to .xyz files, and read them with ASE.

Try carry out the geometry optimization again and compare the results.

EXPLORATION TASK

Please repeat the calculations on other surfaces such as fcc(110), fcc(100), fcc(211), bcc(110), hcp(0001), etc.

Examples:

- Pt(111), Pd(111), Cu(111), Rh(111), Ir(111)
- Ni(110), Ni(100), Ni(211)
- Co(0001), Ru(0001)
- Fe(111)

SOME FURTHER STUDIES

- Saddle point calculations: using nudged elastic band calculation, to find transition states and activation energies of various reactions
- Use graphene or carbon nanotube instead of metal surfaces
- MLIPs are not perfect. You might want to study and compare accuracy of various universal MLIPs for atomistic systems that you are interested in.
- ...